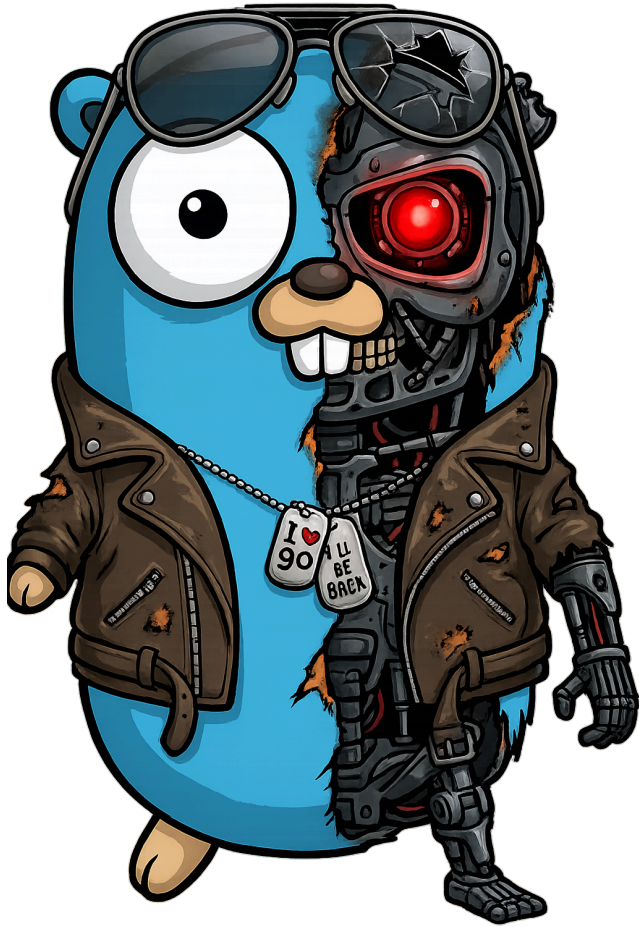




Agentic Engineering

Christian Budde [@MeKo-Christian](#)

Von einfachen Anfragen zu agentischem Arbeiten



KI-Transformation

„Unternehmen, die beim Einführen von KI ihre Prozesse konsequent neu gestalten, übertreffen ihre Umsatzziele doppelt so häufig wie jene, die einfach Tools einführen und auf Wirkung hoffen.“

[Gartner, 2025](#)



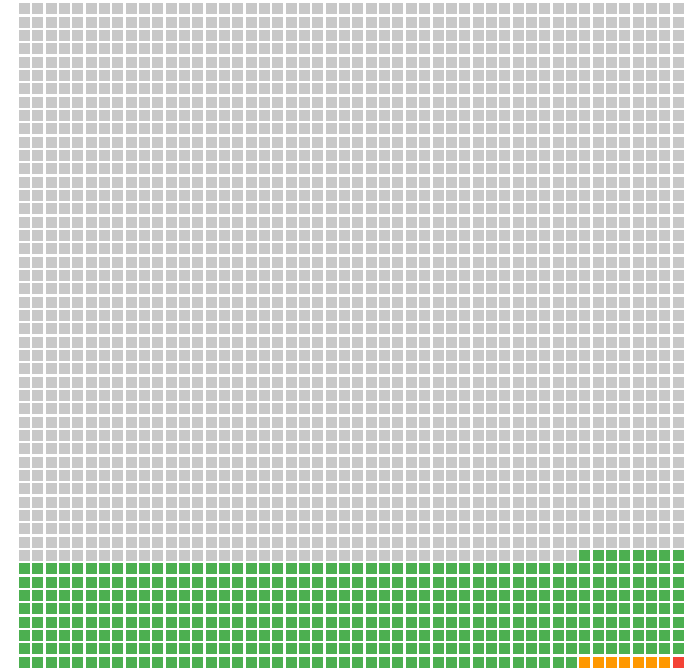
KI-Transformation: Nutzung von AI Tools

- 16% nutzen freie Tools
- 0,3% nutzen Bezahldienste
- 0,04% nutzen Code-Agenten

Quelle: [LinkedIn Post](#)

Jeder Punkt entspricht ~3,2 Millionen Menschen

2.500 Punkte = 8,1 Milliarden Menschen. Farbe = fortschrittlichste KI-Interaktion, Feb 2026.



● Nie AI genutzt · ~6,8 Mrd. (84%) ● Kostenlose Chatbot-Nutzer · ~1,3 Mrd. (16%)
● Beahlt \$20/Monat für KI · ~15–25 Mio. (~0,3%) ● Nutzt Coding Scaffold · ~2–5 Mio. (~0,04%)

Einzelner Prompt

ChatGPT, Claude.ai, Gemini, ...

Mensch: Prompt

LLM: Antwort



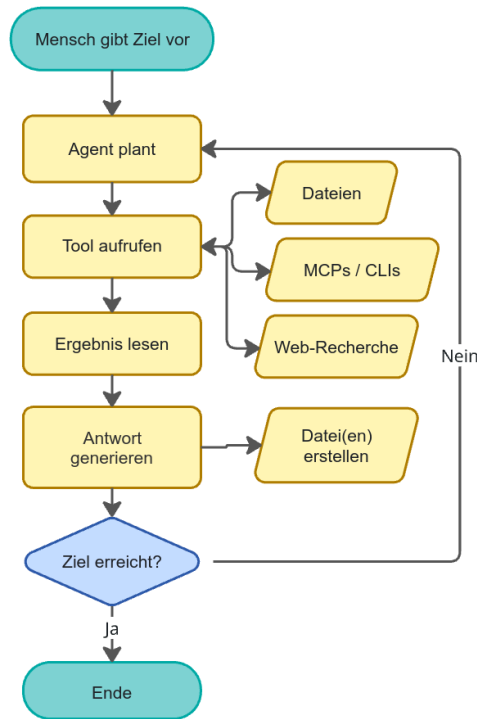
Mensch liest,
kopiert, prüft,
führt aus, ...

- **Eine** Frage, **eine** Antwort
- LLM sieht nur Text — **handelt nicht**
- Jeder Schritt: Mensch ist Ausführender
- Kontext endet mit der Antwort

→ **Prompt Engineering** = Kunst, den richtigen Prompt zu formulieren

Agent

[Claude Code, Codex, Gemini CLI, Cursor, GitHub Copilot Agent, ...]



- **Ein** Ziel, **viele** autonome Schritte
- LLM **handelt**: schreibt, führt aus, liest
- Mensch gibt Richtung — Agent arbeitet
- Kontext bleibt über den ganzen Prozess erhalten

→ **Kontext Engineering** = Kunst, den Agenten mit dem richtigen Kontext zu versorgen

Skills



Skills — Wissen on demand

Wie Neo, der in Sekunden Kung-Fu lernt:
Agenten bekommen Fähigkeiten *injiziert* —
nicht durch Training, sondern durch Kontext.

Was sind Skills?

Was sind Skills?

- Wiederverwendbare Prompt-Bausteine / Anweisungen
- Domänenwissen, Patterns, Coding-Standards
- Tool-Nutzungsanleitungen für den Agenten
- In Claude Code: `/skill`-Dateien im Projekt

Warum wichtig?

- Agent kennt *deine* Konventionen, nicht nur allgemeines Wissen
- Qualität steigt ohne den Agenten neu zu trainieren
- Skills = das neue Institutional Knowledge

Model Context Protocol

Was ist MCP?

Ein offenes Protokoll, das Agenten standardisiert mit externen Tools, Daten und Diensten verbindet.

„USB-C für KI-Agenten“

Das Problem davor:

- Jede App × jedes Modell = eigene Integration
- N×M Custom-Connectoren, kaum wiederverwendbar

Die drei Kern-Primitive:

 **Tools** — Aktionen, die der Agent ausführen kann

z.B. Datei lesen, API aufrufen, DB abfragen

 **Resources** — Daten, die der Agent lesen kann

z.B. Dateisystem, Dokumentation, Codebase

 **Prompts** — Wiederverwendbare Prompt-Templates

z.B. vordefinierte Workflows, Kontext-Snippets

- Anthropic, Nov 2024

Heute:

OpenAI, Google, alle großen Anbieter unterstützen MCP.

Seit Dez 2025 unter der Linux Foundation (AAIF).

Architektur:

Agent (MCP Client)

- ↔ MCP Server A (GitHub, Jira, ...)
- ↔ MCP Server B (Dateisystem, DB, ...)
- ↔ MCP Server C (eigene Tools, ...)

*„Fully give in to the vibes, embrace exponentials,
forget that the code even exists.“*

- Code generieren → **Accept All** → hoffen
- Kein Review, keine Tests, kein Verständnis
- Optimiert für: schnelle Wegwerfprojekte
- Verantwortung: **keine**

Funktioniert für Prototypen und Wegwerfcode —
nicht für Software, die jemand warten, deployen oder auf die sich jemand verlassen muss.

Agentic Engineering

Willison, 2025

*„Seasoned professionals accelerate their work —
proudly accountable for the software they ship.“*

- Agent = **LLM + Tools + Loop** → Ziel erreichen
- Code wird ausgeführt, getestet, iteriert
- Expertise wird verstärkt, nicht ersetzt
- Verantwortung: **bleibt beim Entwickler**

**Die Teilkosten fürs Tippen sinken —
die Kosten für *guten* Code nicht.**

Korrektheit, Tests, Sicherheit, Wartbarkeit:
weiterhin Aufgabe des Entwicklers.

Das Ökosystem: Coding Agents & CLIs

Terminal / CLI

- **Claude Code** — Sonnet 4.6, Opus 4.6)
- **OpenAI Codex CLI** — GPT-5.3-Codex, GPT-5.4,
- **Gemini CLI** — Gemini 3.1 Pro (1M Tokens)

IDE-Erweiterungen

- **GitHub Copilot Agent** — GPT-5, Claude Sonnet
- **Cline / Roo Code** — VS Code, beliebige Modelle
- **Continue** — open source, lokal oder cloud

AI-first IDEs

- **Cursor** — Verschiedene Modelle, Composer
- **Windsurf** (Codeium) — SWE-1, Claude, GPT
- **VS Code + Copilot** — Edits-Modus, Agent-Modus

Cloud / Async Agents

- **Devin** (Cognition) — vollständig autonom
- **Jules** (Google) — Hintergrund-Tasks, GitHub-Integration
- **GitHub Copilot Workspace** — issue → PR
- **SWE-agent** — Forschung, SWE-Bench

Agentic Engineering Patterns

nach Simon Willison



Code ist billig geworden

Früher:

- Einige hundert Zeilen sauberer, getesteter Code = ein voller Arbeitstag
- Features wurden nach Aufwand/ Nutzen-Verhältnis priorisiert
- Refactoring „lohnt sich nicht“

Heute:

- Parallele Agenten: Implementierung, Tests, Doku *gleichzeitig*
- Ein Mensch steuert mehrere Code-Streams
- „Zu aufwendig“ → einfach einen Agenten losschicken

Qualität braucht den Menschen

*„Die Kosten fürs Erzeugen sinken —
die Kosten für Qualität nicht.“*

Code:

- Korrektheit & Fehlerfreiheit
- Tests & Regressionssicherheit
- Wartbarkeit & Einfachheit
- Sicherheit, Performance

Daten & Analysen:

- Stimmen die Zahlen tatsächlich?
- Richtige Formeln, richtige Bezüge?
- Plausibilitäts-Checks durchgeführt?
- Fehlende Daten / Ausreißer erkannt?

Ob Code, Excel-Chart oder Data-Science-Pipeline:
Prüfen bleibt Aufgabe des Menschen.



Sammele, was du kannst

„Hoard things you know how to do“

Das Prinzip:

- Erfahrung = Sammlung bewiesener Lösungen
- Nicht nur *wissen*, sondern *gesehen haben*, dass es funktioniert
- Blog-Posts, TIL-Notizen, Repos, Code-Snippets

Der Schlüssel:

Mit Agenten musst du einen Trick nur **ein einziges Mal** herausfinden.

Rekombination:

Zwei bekannte Lösungen → Agent baut eine neue daraus.

Beispiel:

Tesseract.js (Bild → Text)

1. PDF.js (PDF → Bild)

→ Browser-OCR für PDFs

Agenten durchsuchen deine Sammlung
und kombinieren Bausteine automatisch.

Kumulativer Wissensaufbau

„Compound Engineering“

Projekt A → Was hat gut funktioniert?



Patterns dokumentieren



Projekt B → Agent nutzt die Patterns automatisch



Neue Erkenntnisse ergänzen



Projekt C → Agent wird immer besser

Kumulativer Wissensaufbau — in der Praxis

Was dokumentieren?

- Was lief gut, was nicht?
- Typische Fehler und deren Lösung
- Entscheidungen und deren Begründung

Wo ablegen?

- AGENTS.md — Projekt-Anweisungen
- Skills — wiederverwendbare Prompt-Bausteine
- Projekt-Wiki / README — für das Team

Was bringt das?

- Agent macht denselben Fehler nicht zweimal
- Neue Teammitglieder (und Agenten) starten auf höherem Niveau
- Qualität steigt mit jedem Durchlauf

Beispiel:

Retro Projekt A: „Agent hat immer zu große PRs erstellt.“

→ AGENTS.md: „PRs max. 200 Zeilen“

Projekt B: Agent hält sich daran.

✨ KI soll **bessere** Ergebnisse liefern

*„Schlechtere Ergebnisse mit Agenten zu liefern ist eine **Entscheidung**.
Wir können uns auch für bessere entscheiden.“*

Exploratives Prototyping:

- Hypothesen schnell durch Prototypen prüfen
- Simulationen & Lasttests in Minuten aufsetzen
- „Senkt die Kosten für Experimente auf fast null“

Daten & Analysen:

- Excel-Auswertungen automatisch erstellen
- Diagramme in verschiedenen Varianten testen
- Daten bereinigen und aufbereiten lassen
- Mehr Zeit zum Nachdenken, weniger zum Formatieren

Ideale Code-Aufgaben für Agenten

Refactoring:

- API-Redesigns über Dutzende Stellen
- Benennungs-Inkonsistenzen bereinigen
- Doppelte Funktionalität konsolidieren
- Große Dateien modularisieren

*Agenten im Hintergrund laufen lassen —
Ergebnis als Pull Request reviewen.*

Qualitäts-Hebel:

- Test-Coverage erhöhen
- Dokumentation aktualisieren
- Typ-Annotationen ergänzen
- Dependency-Updates durchführen
- Sicherheits-Audits automatisieren

*Aufgaben, die sich „nie lohnen“ —
jetzt im Hintergrund erledigt.*

Git — Versionskontrolle für Code

Das Fundament moderner Softwareentwicklung

Was ist Git?

Ein System, das *jede Änderung* an Dateien aufzeichnet — wie eine unbegrenzte Undo-History für das gesamte Projekt.

- **Commit** — ein Snapshot des aktuellen Stands
- **Branch** — ein paralleler Arbeitsstrang
- **Merge** — Änderungen zusammenführen
- **Pull Request** — Änderungen zur Review vorlegen

Warum Git wichtig ist:

- Nichts geht verloren — jeder Stand ist wiederherstellbar
- Mehrere Personen können gleichzeitig arbeiten
- Änderungen sind nachvollziehbar (wer, wann, warum)
- Standard in der gesamten Branche

→ *Sicherheitsnetz* für jede Codeänderung

Git + Agenten

Agenten sprechen fließend Git

Grundlagen delegieren:

- „*Commit diese Änderungen*“ → aussagekräftige Messages
- „*Zeig mir die Änderungen von heute*“ → `git log`
- „*Räum dieses Git-Chaos auf*“ → Merge-Konflikte lösen

*Agenten arbeiten immer auf einem Branch —
main bleibt geschützt.*

Commit-History als erzählte Geschichte

Agenten helfen beim Kuratieren:

- Commits zusammenfassen oder aufteilen
- Messages verbessern
- Dateien aus Commits entfernen
- Code in neue Repos extrahieren

→ *Strategie* statt Syntax

❌ Anti-Pattern: Ungeprüfter Code im PR

*„Stelle keine Pull Requests mit Code,
den du nicht selbst reviewed hast.“*

Das Problem:

- Hunderte Zeilen Agent-Code → „Accept All“ → PR
- Reviewer hätten den Agent selbst prompten können
- Verantwortung wird auf andere abgewälzt

Gute Agent-PRs haben:

1. **Funktionalität** — Code funktioniert nachweislich
2. **Scope** — Klein genug für effizientes Review
3. **Kontext** — Ziel, Issue-Links, Spezifikation
4. **Eigene Prüfung** — Test-Notizen, Screenshots

Subagenten: Parallele Arbeit

Das Problem:

Kontext-Fenster ist begrenzt
(ca. 200k Tokens) → Agent vergisst

Die Lösung:

Agent startet frische Kopien von sich selbst —
jeweils mit eigenem Kontext-Fenster.

Haupt-Agent

```
├→ Subagent A (Codebase erkunden)
├→ Subagent B (Tests schreiben)
└→ Subagent C (Doku aktualisieren)
    ↓ parallel ↓
Ergebnisse zurück an Haupt-Agent
```

Spezialisierung:

-  **Explorer** — Codebase analysieren
-  **Testläufer** — Fehler isolieren
-  **Reviewer** — Bugs & Design-Schwächen finden
-  **Debugger** — Ursachen untersuchen

Performance:

- Parallel auf günstigeren Modellen
- Haupt-Agent behält wertvollen Kontext

Red/Green TDD mit Agenten

„Angenehm kurz und effektiv“ — Willison

- **Red:** Tests schreiben → bestätigen, dass sie *fehlschlagen*
- **Green:** Code implementieren → Tests bestehen
- **Refactor:** Aufräumen, vereinfachen

Prompt:

*„Baue eine Python-Funktion die X tut.
Nutze Red/Green TDD.“*

Warum besonders wichtig mit Agenten:

- Verhindert nicht-funktionierenden Code
- Schützt vor unnötigen Features
- Baut robuste Test-Suite auf
- Regressionssicherheit wächst automatisch

Achtung:

Red-Phase nie überspringen!

„Erst die Tests laufen lassen“

*„Automatisierte Tests sind nicht mehr optional
wenn man mit Coding-Agenten arbeitet.“*

Die Vier-Wort-Regel:

„First run the tests“

Jede Session mit dem Agenten so
beginnen.

Alternativ: „*Run ‘just test’*“

Drei Effekte:

1. **Discovery** — Agent findet & versteht die Test-Suite
2. **Kontext** — Test-Output zeigt Projektumfang & Komplexität
3. **Mindset** — Agent entwickelt Test-fokussierte Arbeitsweise



Manuelles Testen mit Agenten

*„Gehe nie davon aus, dass LLM-generierter Code funktioniert,
bis er **ausgeführt** wurde.“*

Wie Agenten manuell testen:

- Python: `python -c "..."` direkt im Terminal
- Web-APIs: Dev-Server starten + `curl` erkunden
- Browser: Playwright, agent-browser

Der Kreislauf:

Agent generiert Code



Agent testet manuell



Problem gefunden?

→ Fix mit Red/Green TDD

→ Test bleibt als Regression



Qualität steigt dauerhaft

Prüfen ist Pflicht — nicht nur beim Code

„Vertrauen ist gut — Kontrolle ist bei KI-Ergebnissen unverzichtbar.“

Daten & Analysen:

- Stimmen die Zahlen in der Excel-Tabelle?
- Sind Formeln und Bezüge korrekt?
- Fehlen Datensätze oder gibt es Ausreißer?
- Wurde die richtige Spalte aggregiert?

Texte & Dokumente:

- Fakten geprüft? (LLMs halluzinieren!)
- Quellen vorhanden und korrekt?
- Fachbegriffe richtig verwendet?

Prüfen ist Pflicht — nicht nur beim Code

Präsentationen & Berichte:

- Passen Diagramm und Aussage zusammen?
- Sind Einheiten und Achsenbeschriftungen richtig?
- Stimmt die Schlussfolgerung mit den Daten überein?

Grundregel:

Je wichtiger die Entscheidung,
desto gründlicher die Prüfung.

**KI beschleunigt das Erstellen —
nicht das Prüfen.**

Code verstehen: Walkthroughs & Visualisierungen

Linear Walkthroughs:

Agent erstellt strukturierte Erklärungen des gesamten Codes — Datei für Datei.

„Lies den Quellcode und erstelle einen detaillierten Walkthrough. Zeige Code-Snippets mit grep/cat.“

→ Auch Vibe-Coding-Projekte werden nachträglich verständlich.

Interaktive Erklärungen:

Agent baut animierte HTML-Visualisierungen, die Algorithmen Schritt für Schritt zeigen.

- Platzierungs-Algorithmen animiert
- Kollisions-Erkennung sichtbar gemacht
- Abstrakte Konzepte → interaktiv erforschbar

Cognitive Debt vermeiden

*„Verstehen, **was** der Agent gebaut hat —
nicht nur, **dass** es funktioniert.“*

Das Problem:

- Agent erzeugt Code schneller als man lesen kann
- Projekt wächst, aber das Verständnis nicht
- Kleine Änderungen werden riskant, weil niemand die Zusammenhänge mehr überblickt

Die Lösung:

**KI kann nicht nur bauen —
sie kann auch erklären.
Beides nutzen!**

- Architektur-Entscheidungen dokumentieren
- Visualisierungen anfordern

Willisons Patterns — Zusammenfassung

Prinzipien

1. Code ist billig — *guter* Code nicht
2. Sammle bewiesene Lösungen
3. Nutze KI für *besseren* Code
4. Kein ungeprüfter Code in PRs

Arbeiten mit Agenten

1. Git als Sicherheitsnetz nutzen
2. Subagenten für parallele Arbeit
3. Kontext-Fenster bewusst managen

Testing & QA

1. Red/Green TDD — immer
2. „Erst die Tests laufen lassen“
3. Manuelles Testen nicht vergessen

Code verstehen

1. Lineare Walkthroughs generieren
2. Interaktive Visualisierungen bauen
3. Cognitive Debt vermeiden

Quelle: simonwillison.net/guides/agentic-engineering-patterns

Wie geht es weiter?

Tool-Lifecycle: Vom Experiment zum Unternehmenstool

Der typische Weg:

1. Experimentieren
- ↓
2. Wissen aufbauen
- ↓
3. Wiederholen
- ↓
4. Tool schreiben

LLM + MCP ausprobieren → bewährte Aufrufe
→ parametrisierbares Script

Wenn das Tool wächst:

5. Standalone-Tool
- ↓
6. Teilen
- ↓
7. Verteilen
- ↓
8. Software-Engineering

Klar definiert, deploybar → Kollegen nutzen mit
→ Server statt Executables → klassische
Prinzipien

→ Stufen 1–4 entstehen beim Experimentieren. Was überlebt, wächst weiter.



Was Tools überleben lässt

„Software überlebt, wenn sie mehr Kognition spart als sie kostet.“

Steve Yegge, *Software Survival 3.0*

Was Tools am Leben hält:

- **Komprimiertes Wissen** — löst, was schwer zu replizieren ist
- **Breite Nutzbarkeit** — amortisiert Lernaufwand
- **Geringe Friction** — funktioniert wie erwartet

Was Tools sterben lässt:

- **Hohe Awareness-Kosten** — schwer zu entdecken oder zu lernen
- **Hohe Friction-Kosten** — Fehler, Missverständnisse
- **Enge Nutzungsszenarien** — zu spezifisch

→ Agenten synthetisieren lieber selbst

→ „*Make something crazy to re-synthesize*“

⚙️ Substrate-Effizienz: Compute auf dem richtigen Substrat

„Nobody is coming for grep.“

Steve Yegge, *Software Survival 3.0*

Das Prinzip:

LLM-Inferenz ist das teuerste Rechensubstrat.

Tools, die Arbeit auf CPUs, Datenbanken oder Kalkulatoren verlagern, sparen Token per Definition.

Beispiele — günstig schlägt intelligent:

- grep — Mustererkennung auf CPU
- Taschenrechner — Arithmetik ohne Inferenz
- Datenbankabfragen — strukturierter Datenzugriff
- ImageMagick, Parser — komplexe Transformationen

→ „*Pattern matching über Text: CPU schlägt GPU um Größenordnungen.*“

→ Genau das macht MCP-Tools wertvoll:
Echte Daten ohne LLM-Rekonstruktion

Die Skalierungsfalle: Teilen ist ein Engineering-Problem

Einzelner Nutzer:

- Eigene Annahmen, eigene Umgebung, eigene Daten
- Fehler = eigenes Problem
- Keine Distribution nötig

Im Team geteilt:

- Verschiedene Anforderungen, Umgebungen, Annahmen
- Bugs werden sichtbar — und wichtig
- Distribution und Versionierung nötig
- Sicherheit, Logging, Fehlerbehandlung

→ Klassisches Software-Engineering kehrt zurück

Die Patterns dieses Talks — TDD, Git als Sicherheitsnetz, Kontext-Management — sind kein Zufall.

Evolution oder Redundanz im Tool-Ökosystem?

Die Realität im Team:

- Mehrere entwickeln ähnliche Tools unabhängig
- „Survival of the fittest“ → Redundanz
- Agenten **können** helfen:
zusammenführen,
umschreiben, das Beste kombinieren

→ Aber der Engpass wird schnell:

Kommunikation & Anforderungsengineering, nicht das Coden selbst

Die richtige Haltung:

- Skepsis gegenüber eigenen Tools ist genauso wichtig wie gegenüber KI-Ausgaben
- Wegwerfprototyp oder Grundstein? Beides ist valide — wenn bewusst entschieden

Was als Prototyp entsteht, könnte der Grundstein sein —

oder der nächste Wegwerfprototyp.
Entscheidet bewusst.

Appendix

Was sind Token?

Grob kann man sich Token wie Silben vorstellen, allerdings sind typische Wörter der deutschen Sprache auch dem Sprachmodell bekannt und gelten als ein Wort. Bei Rechtschreibfehlern wird es aber schnell wild.

Eine hilfreiche Faustregel lautet, dass ein Token bei gewöhnlichem englischem Text in der Regel etwa vier Zeichen entspricht. Das entspricht etwa drei Vierteln eines Wortes (also 100 Token ~ 75 Wörter).

Sprachmodelle zerlegen Text in **Tokens** – häufig vorkommende Zeichenfolgen, grob vergleichbar mit Silben. Geläufige Wörter werden meist als einzelner Token erkannt; Tippfehler zwingen das Modell, ein Wort in mehrere Fragmente aufzuspalten. Faustregel: 1 Token $\approx$ 4 Zeichen, 100 Tokens $\approx$ 75 Wörter.

Was sind Token? – Übersetzung in Zahlen

[38, 23858, 6250, 873, 3660, 17951, 5700, 10344, 3128, 108099, 11, 42489, 5029, 169732, 191218, 1227, 52542, 89476, 4174, 2019, 129743, 159155, 40917, 844, 101724, 3075, 1605, 63983, 13, 25663, 14753, 18462, 332, 33952, 2302, 45415, 77, 6557, 878, 8267, 28730, 12333, 364, 49500, 133376, 7509, 142978, 88591, 170969, 11, 6021, 1605, 17951, 5536, 185305, 119161, 82821, 125847, 4564, 306, 1227, 43662, 35857, 16832, 101229, 123623, 13, 9286, 123623, 35857, 28072, 148631, 10290, 17003, 63983, 268, 350, 32038, 220, 1353, 17951, 6574, 220, 3384, 191218, 741]

Intern kennt ein Sprachmodell keine Buchstaben – nur Zahlen. Jeder Token wird einer eindeutigen ganzen Zahl zugeordnet. Was wir als lesbaren Text wahrnehmen, ist für das Modell eine Sequenz von Indizes in einem riesigen Vokabular – der eigentliche Eingabevektor, mit dem alle Berechnungen beginnen.

Token-Preise im Vergleich (*per 1M Tokens*)

Modell	Anbieter	Input /1M	Output /1M	Kontext
Claude Opus 4.7	Anthropic	\$5.00	\$25.00	200K
Claude Sonnet 4.6	Anthropic	\$3.00 / \$6.00 ≤ / > 200K Tokens	\$15.00 / \$22.50 ≤ / > 200K Tokens	1M
GPT-5.4	OpenAI	\$2.50 / \$5.00 < / > 272K Tokens	\$15.00 / \$22.50 < / > 272K Tokens	400K
GPT-5.5	OpenAI	\$5.00 / \$10.00 < / > 272K Tokens	\$30.00 / \$45.00 < / > 272K Tokens	—
GPT-5.5 Pro	OpenAI	\$30.00	\$180.00	—

Cache-Reads/-Writes sowie Batch-Rabatte (50%) nicht dargestellt. · Preise Stand April 2026.

Prompt Caching — was steckt dahinter?

Viele Anfragen an ein LLM enthalten einen langen, gleichbleibenden Teil – etwa einen ausführlichen System-Prompt, eine große Dokumentation oder einen langen Gesprächsverlauf. Ohne Caching wird dieser Block bei jeder Anfrage neu berechnet und vollständig in Rechnung gestellt.

Wie Prompt Caching funktioniert:

Das Modell speichert den verarbeiteten Zustand eines Prompt-Abschnitts zwischen. Folge-Anfragen, die denselben Prefix enthalten, lesen diesen Zustand aus dem Cache – statt ihn neu zu berechnen. Das spart sowohl Zeit als auch Kosten.

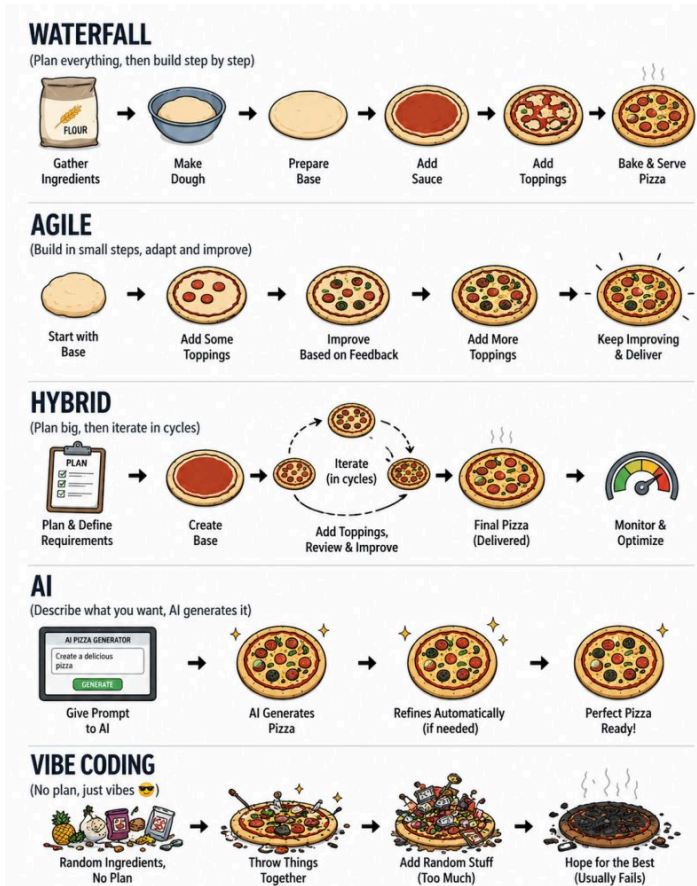
Drei Preiskategorien im Vergleich:

- **Input (normal)** — voller Preis, keine Wiederverwendung
- **Cache Write** — 25 % teurer als normaler Input; der Prefix wird einmalig im Cache abgelegt

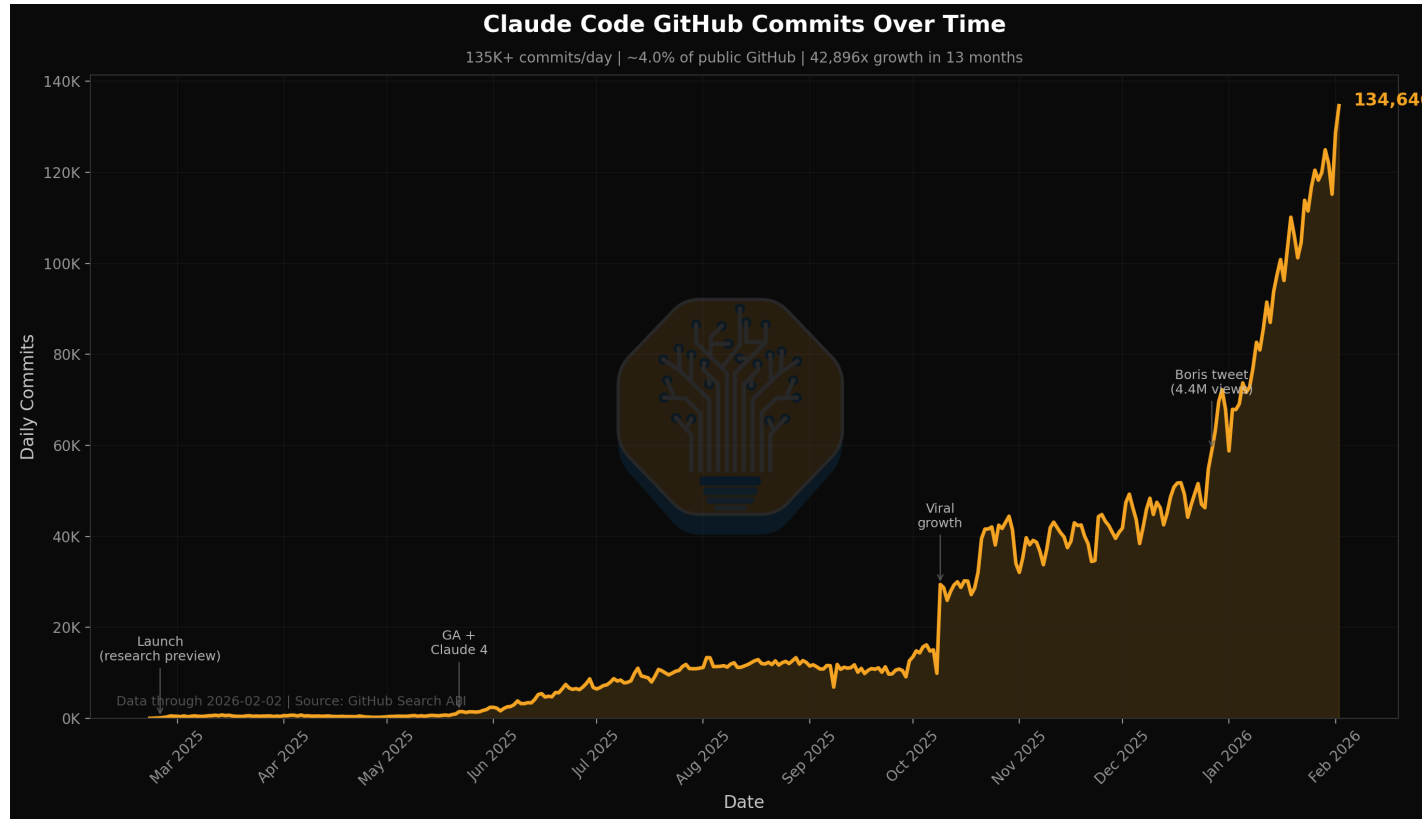
- **Cache Read** — 90 % günstiger als normaler Input; nachfolgende Anfragen lesen aus dem Cache

Lohnt sich ab ca. 2–3 Wiederverwendungen desselben Prefixes. Ideal für System-Prompts, RAG-Dokumente oder lange Gesprächsverläufe.

Entwickeln mit KI im Vergleich zu klassischer Software-Entwicklung



Claude Code: GitHub Commits



Claude Code: Der Wendepunkt

Quelle: [SemiAnalysis – „Claude Code Is The Inflection Point“](#)

- **4% aller öffentlichen GitHub-Commits** werden aktuell von Claude Code erstellt – Prognose: über 20% bis Ende 2026
- Führende Entwickler (Karpathy, Dahl) sehen manuelle Code-Erstellung als zunehmend obsolet – Ingenieure definieren Ziele, KI-Agenten implementieren
- Das READ-THINK-WRITE-VERIFY-Muster ist auf über 1 Mrd. Informationsarbeiter anwendbar (Finanzen, Recht, Consulting)



Der Pelikan-Benchmark

Was wird gemessen?

Simon Willison bittet jedes Modell, per Prompt ein SVG zu erzeugen:

„Generate an SVG of a pelican riding a bicycle.“

Warum das interessant ist:

- Kein Trainingsdatensatz enthält viele Pelikan-auf-Fahrrad-SVGs
- Räumliches Denken + Code-Generierung kombiniert

Was es misst:

- Räumliches & physikalisches Verständnis
- Fähigkeit, *ausführbaren* Code zu erzeugen
- Kreativität vs. mechanische Korrektheit
- Fortschritt über Modellgenerationen hinweg

Ursprung:

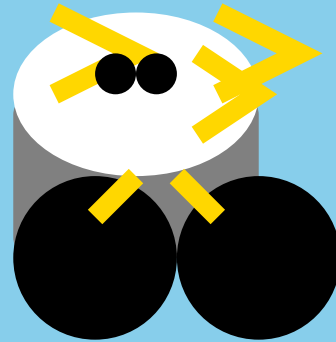
Simon Willison, 2024 —

- Fahrrad korrekt zu zeichnen ist überraschend schwer
- Ergebnis sofort visuell verständlich — kein Rubric nötig

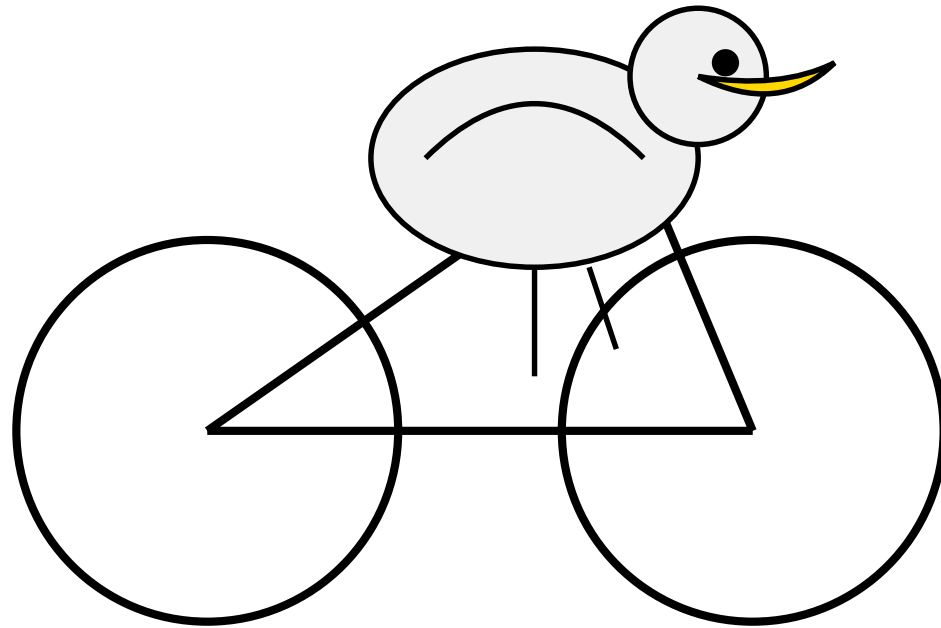
*„I started it as a joke,
but it's actually starting to be a bit useful.“*

github.com/simonw/pelican-bicycle

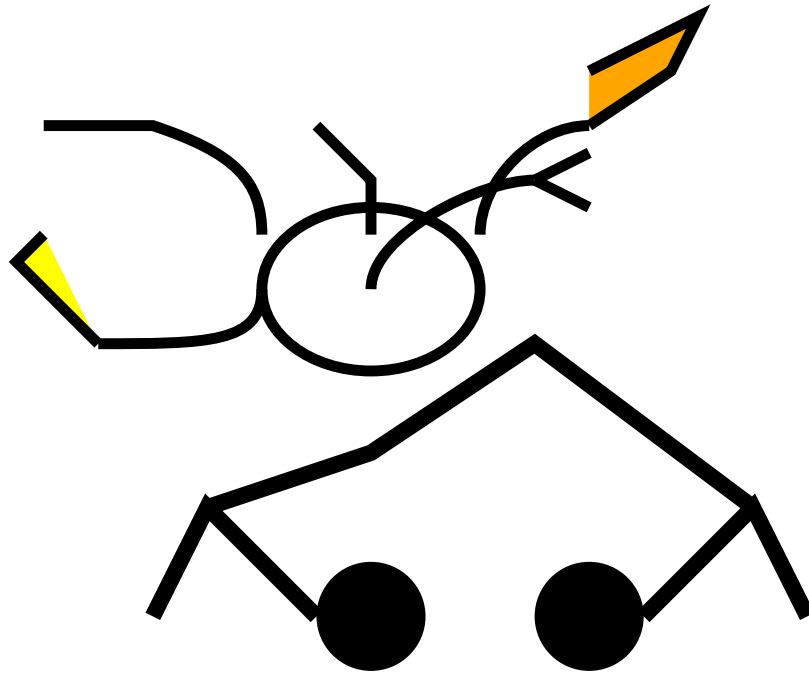
cerebras-llama3.1-70b (2024)



claude-3-5-sonnet (2024)



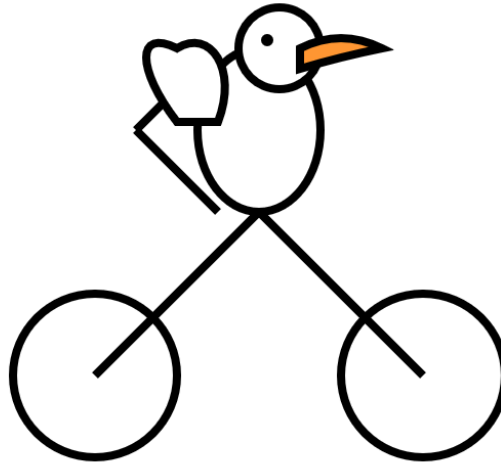
gemini-1.5-pro (2024)



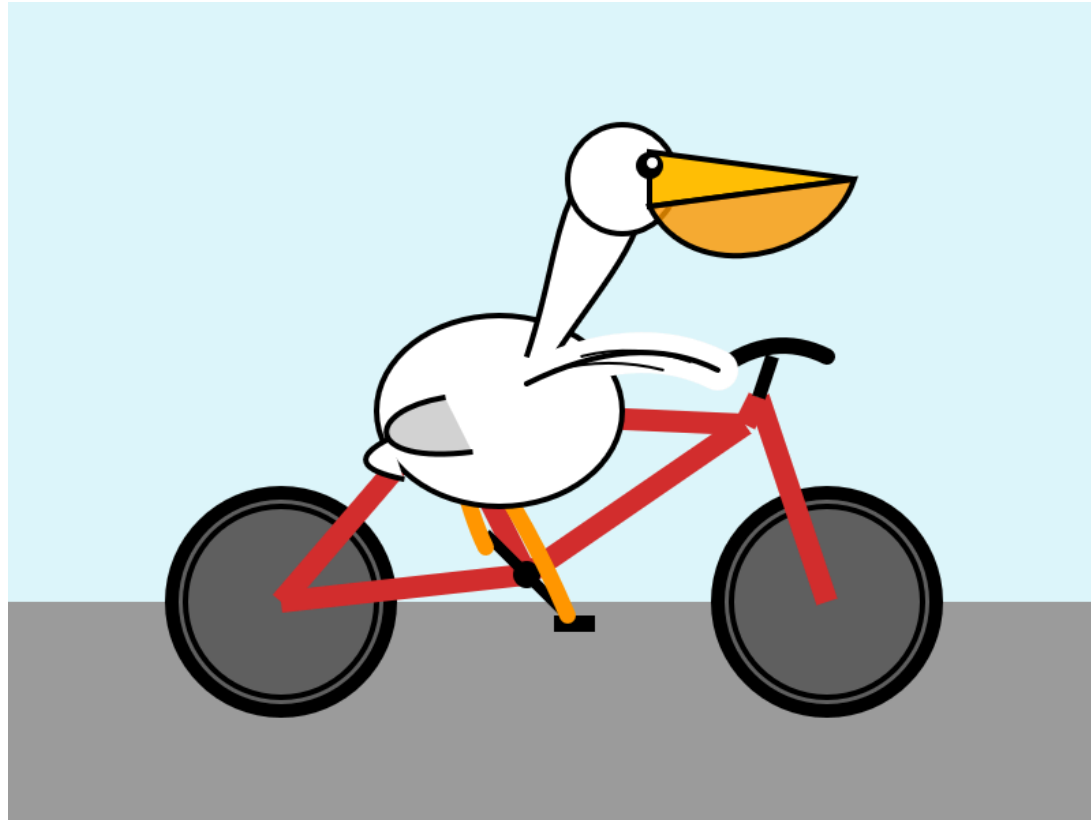
GPT-4o (2024 - “thinking”)



o1 Pro (High - “thinking”)



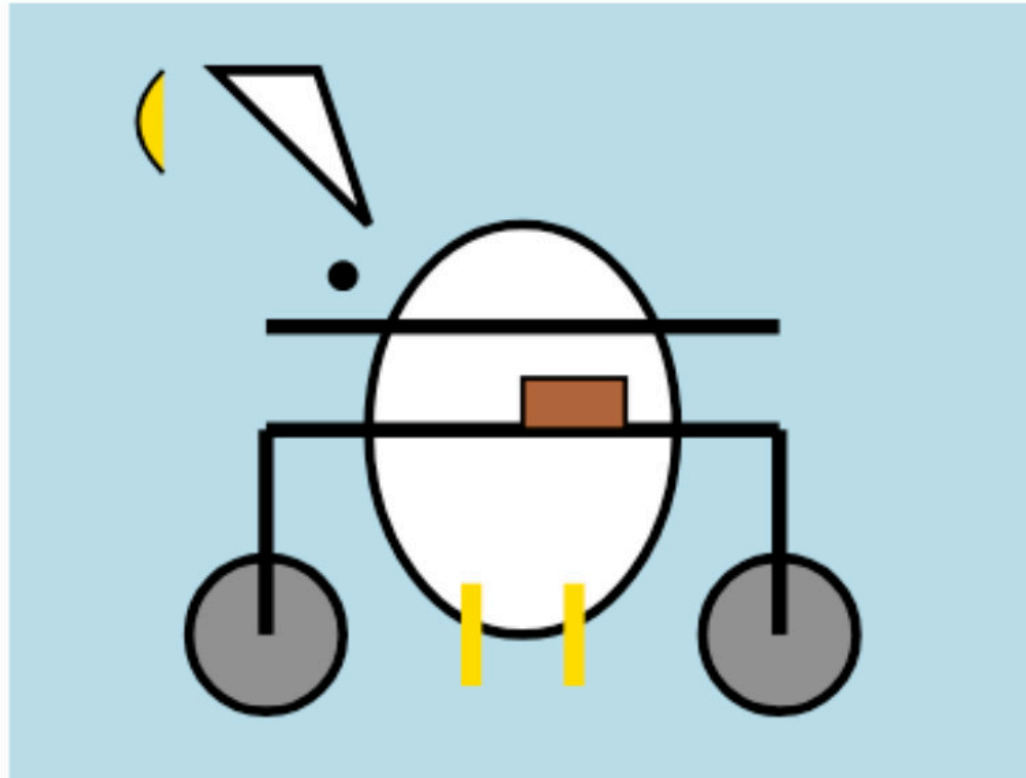
Deep Think



Gemini Flash Thinking



Gemma 3



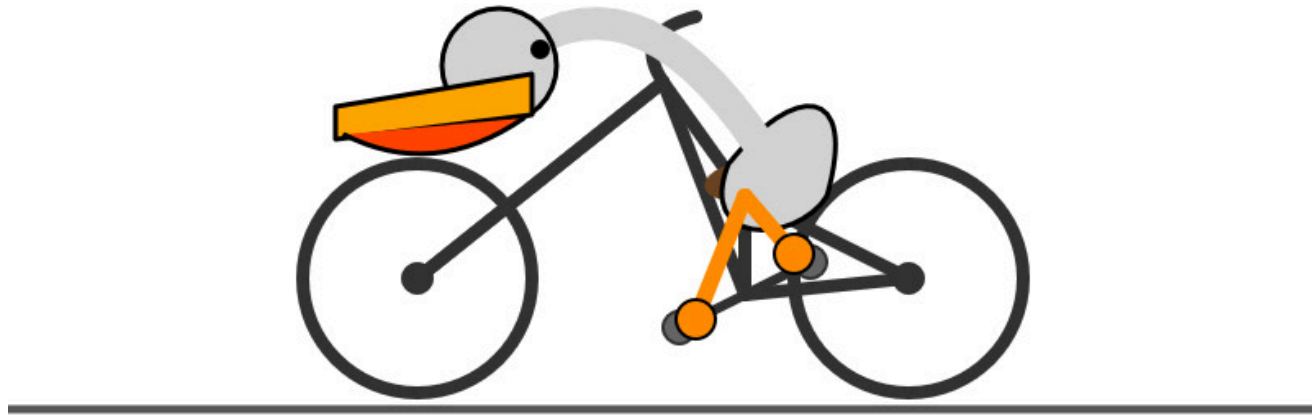
GPT-4.1 (1M Kontext-Tokens)



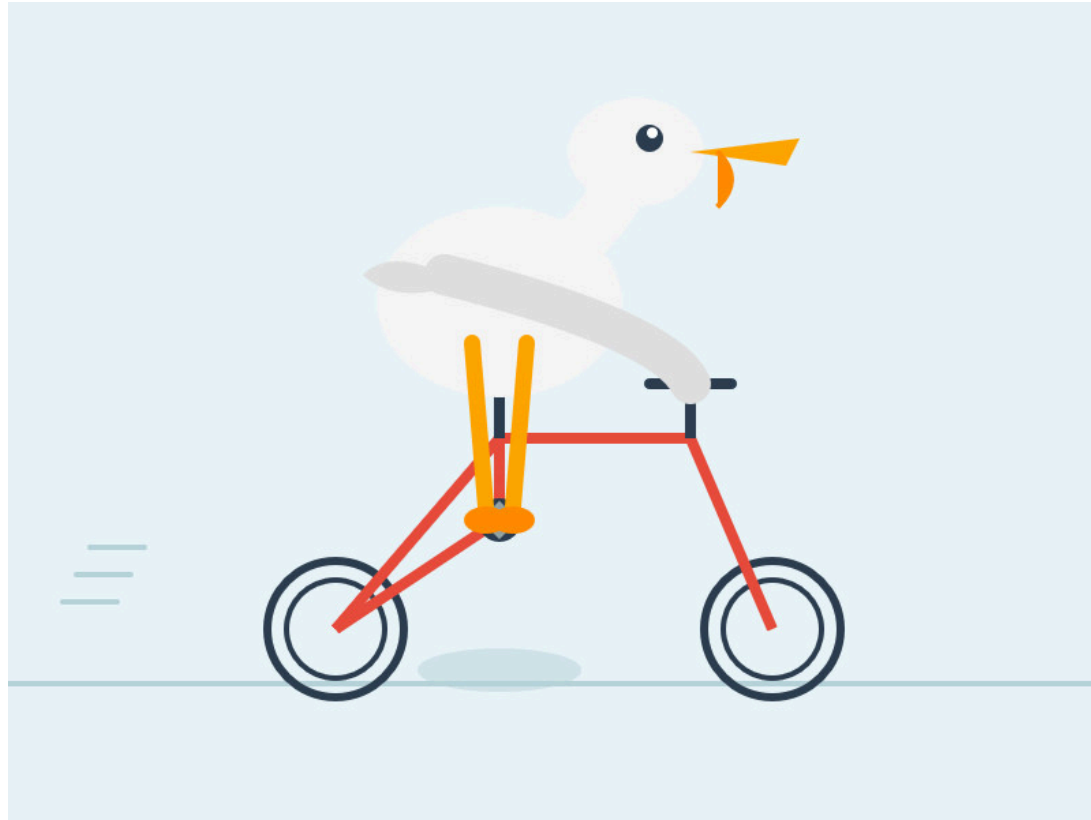
Gemini (latest)



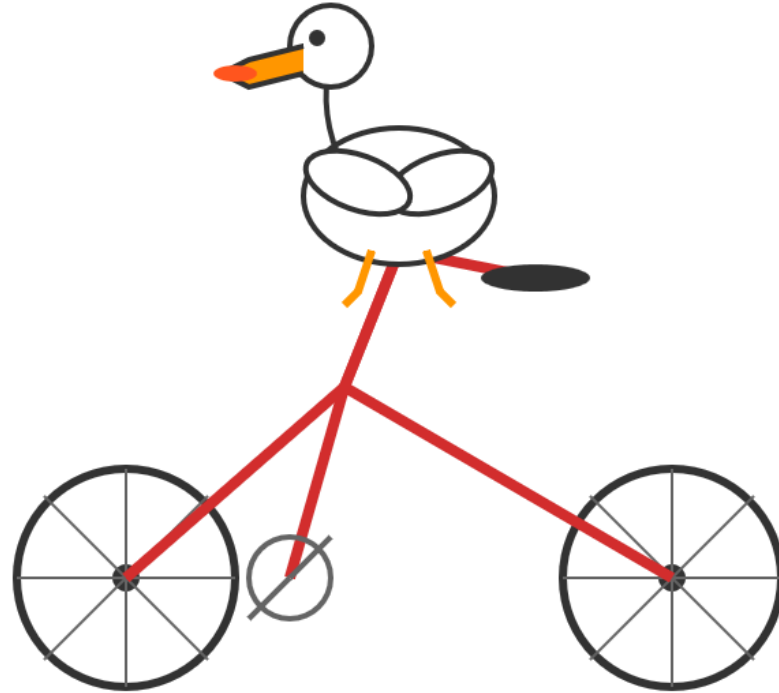
Gemini 2.5 Flash Thinking Max



GLM-4.5



Kimi K2 (Open Source)



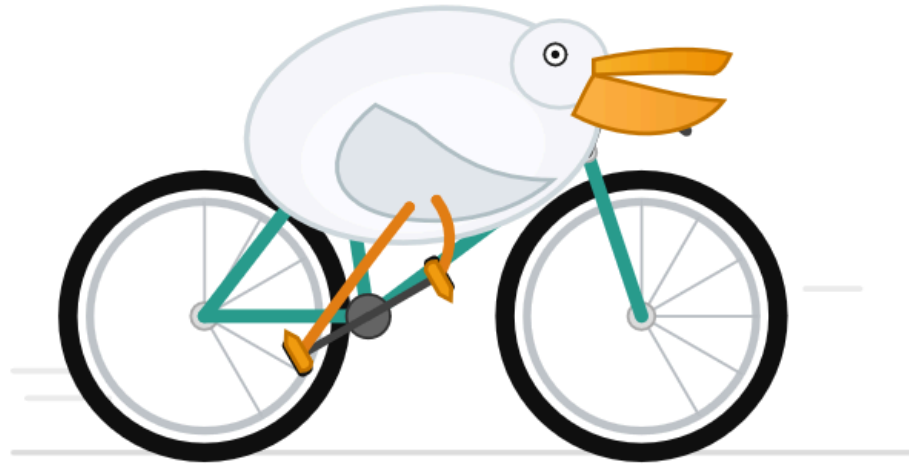
GPT-5



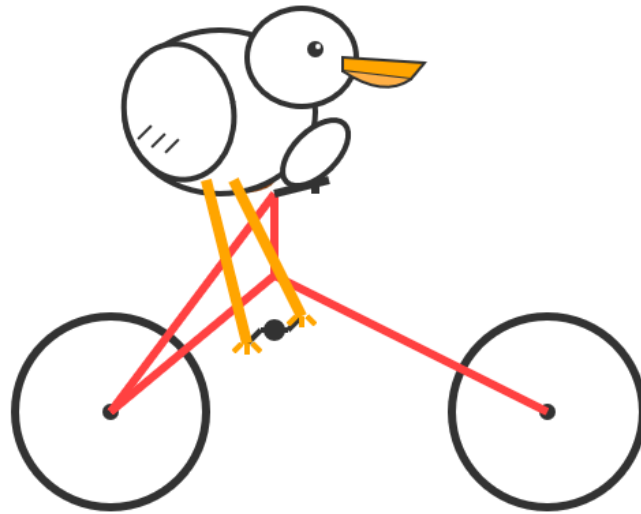
GPT-5 Codex API



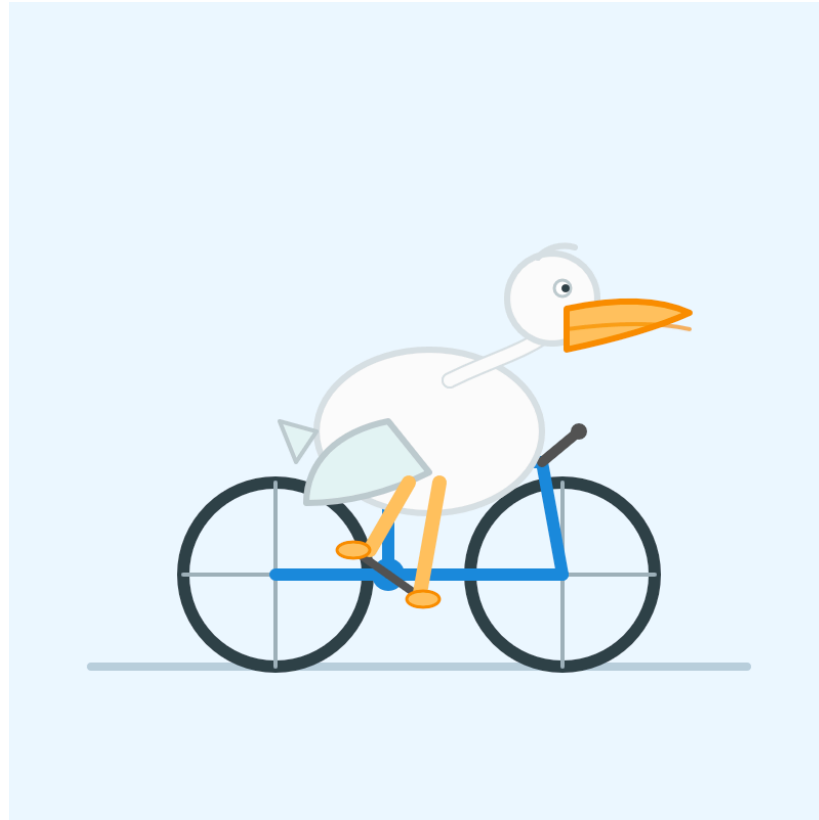
GPT-5 Pro



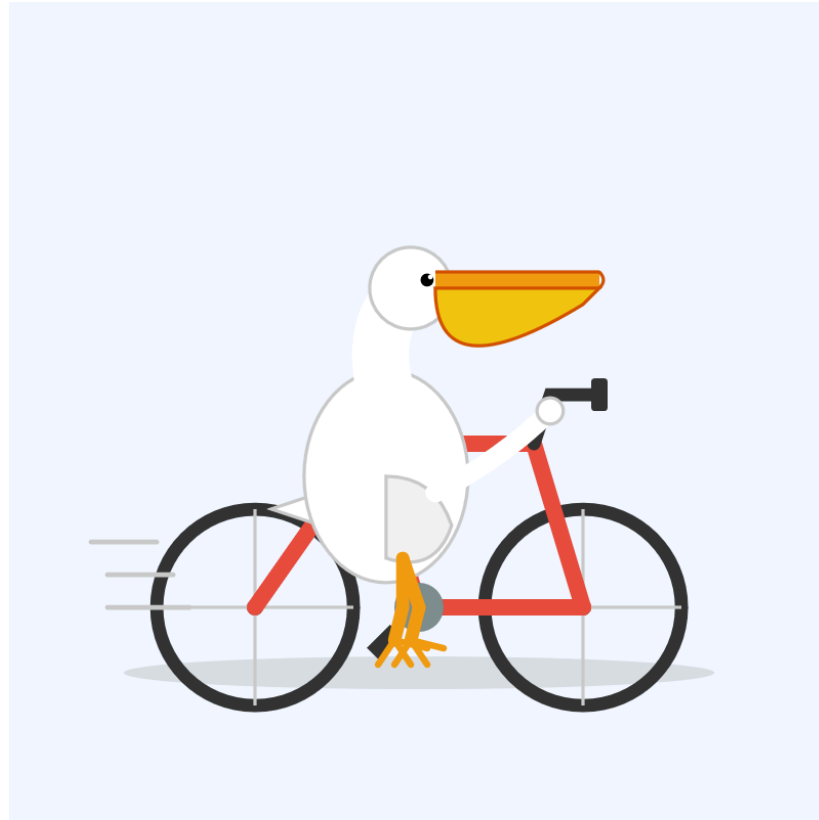
Claude Opus 4.1



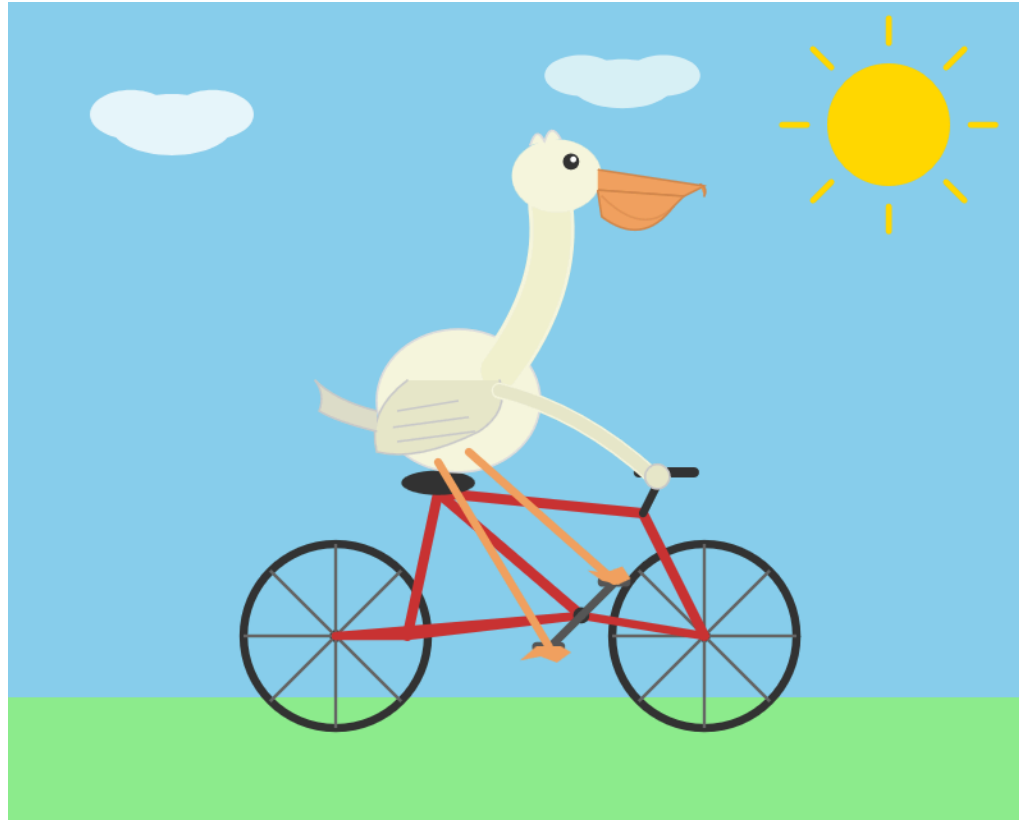
GPT-5.1 High



Qwen 3.5-397B



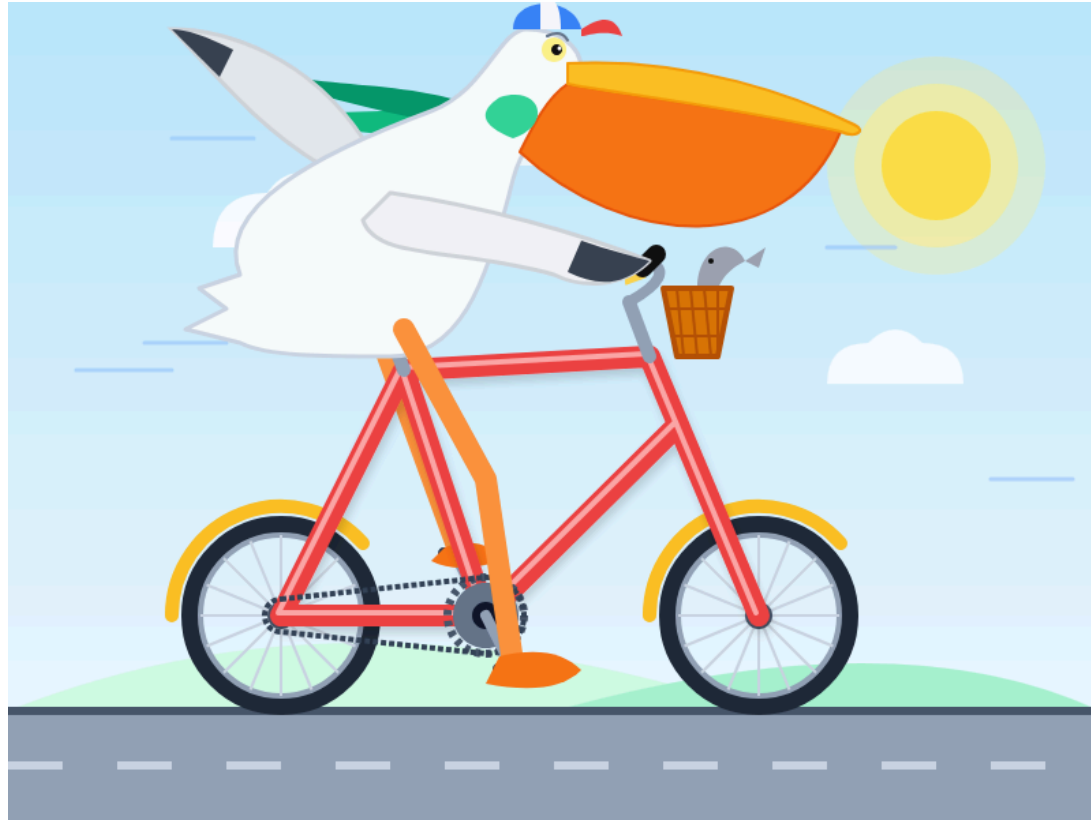
Claude Opus 4.6



Gemini 3 Deep Think



Gemini 3.1 Pro



GLM 5.1



Quellen & Credits

Pelikan-Bilder und inhaltliche Grundgedanken (Agentic Engineering, Vibe Coding, Pelikan-Benchmark) entnommen aus:

simonwillison.net

Simon Willison's Weblog